- ***4*** Starting epsilon for epsilon-greedy policy
- ***5*** Ending/minimum epsilon
- ***6*** Prioritized-replay; duplicates highly informative experiences in the replay this many times
- ***7*** Updates the target network every 25 steps
- ***8*** The total number of parameters for Dist-DQN
- ***9*** Randomly initializes parameters for Dist-DQN
- ***10*** Initializes parameters for target network
- ***11*** Stores each win (successful freeway crossing) as a 1 in this list

These are all the settings and starting objects we need before we get to the main training loop. All of it is roughly the same as what we did for the simulation test, except we have a prioritized replay setting that controls how many copies of a highly informative experience (such as a win) we should add to the replay. We also use an epsilon-greedy strategy, and we will start with an initially high epsilon value and decrease it during training to a minimum value to maintain a minimal amount of exploration.

Listing 7.14. The main training loop

```
from random import shuffle
for i in range(epochs):
    pred = dist_dqn(state,theta,aspace=aspace)
    if i < replay_size or np.random.rand(1) < eps:          1
        action = np.random.randint(aspace)
    else:
        action =
get_action(pred.unsqueeze(dim=0).detach(),support).item()
    state2, reward, done, info = env.step(action)           2
    state2 = preproc_state(state2)
    if reward == 1: cum_rewards.append(1)
    reward = 10 if reward == 1 else reward                  3
    reward = -10 if done else reward                        4
    reward = -1 if reward == 0 else reward                  5
    exp = (state,action,reward,state2)                      6
    replay.append(exp)                                      7

    if reward == 10:                                        8
        for e in range(priority_level):
            replay.append(exp)

    shuffle(replay)
    state = state2
```